

Creating Databases and Database Files



Creating Databases and Database Files

- Creating Databases
- Creating Filegroups
- Creating Schemas
- Creating Database Snapshots

Creating Databases



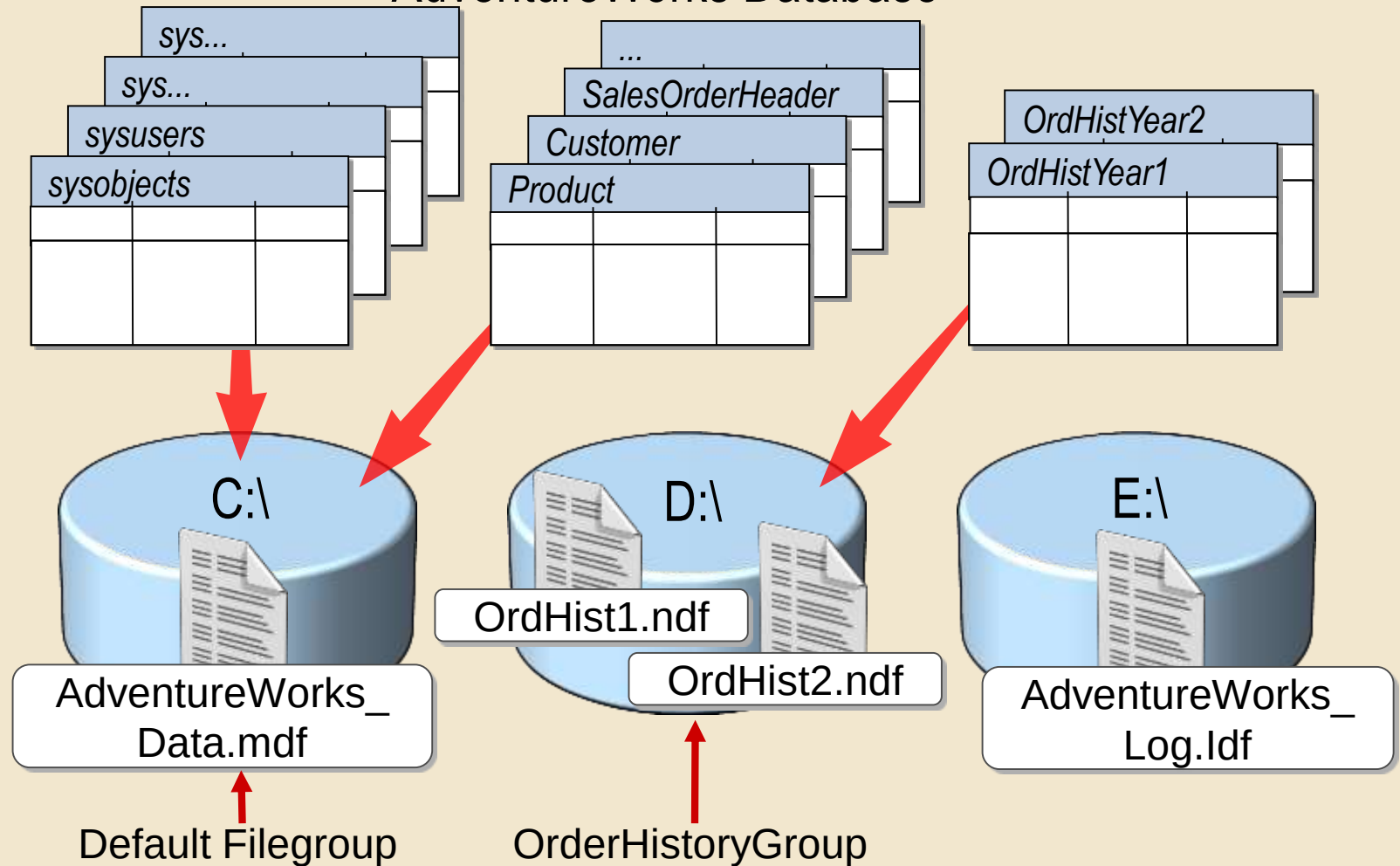
- ❑ Considerations for Creating a Database
- ❑ Transaction Logging
- ❑ Database Options
- ❑ Data Compression

Creating Filegroups

- What are Filegroups?
- Improving Database Performance by Using Filegroups

What are Filegroups?

AdventureWorks Database



Improving Database Performance by Using Filegroups

Create files or filegroups on as many different available local physical disks as possible

Put objects that compete heavily for space in different filegroups

Put different tables used in the same join queries in different filegroups

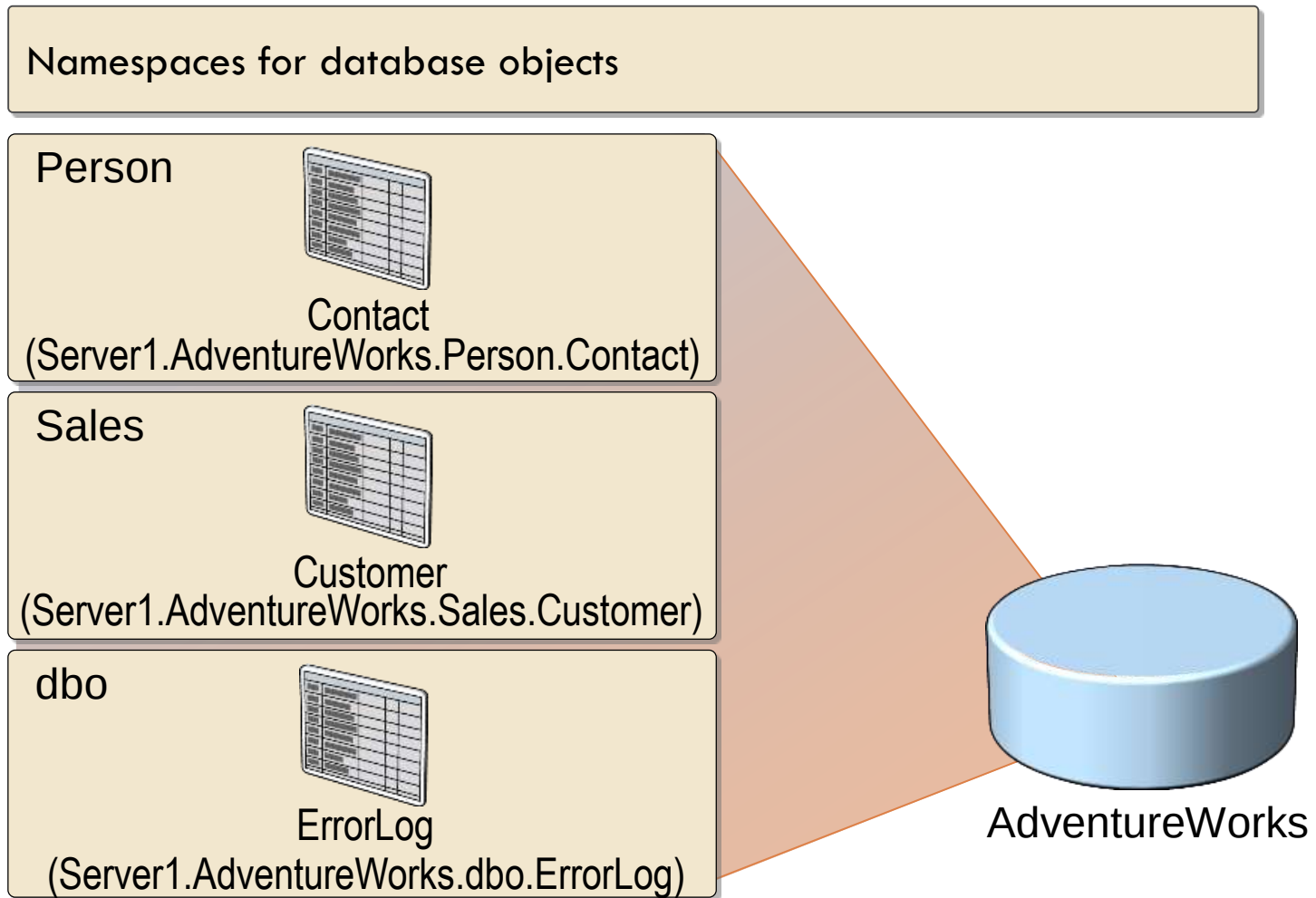
Put heavily accessed tables and the nonclustered indexes that belong to those tables on different filegroups

Creating Schemas

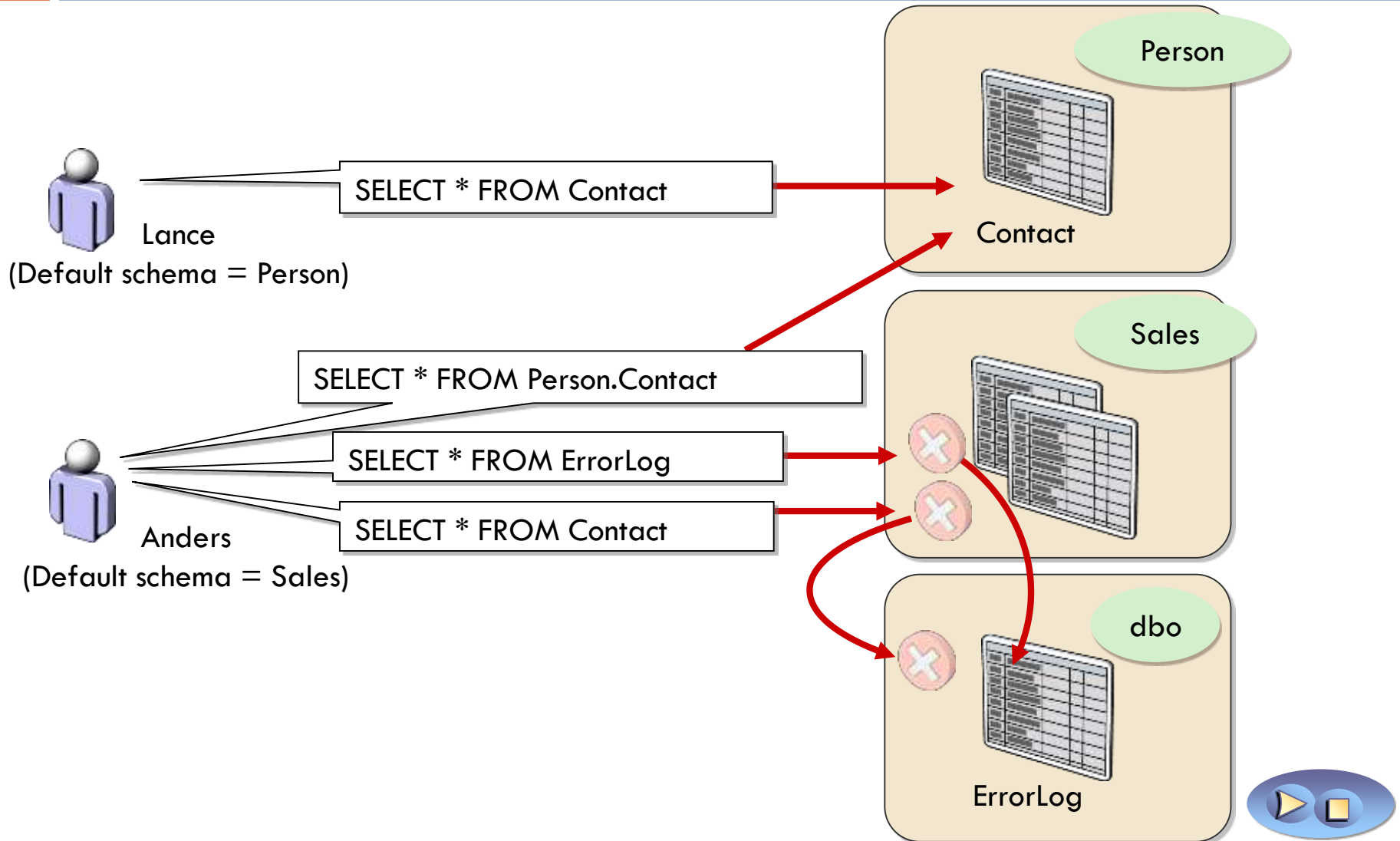


- What are Schemas?
- How Object Name Resolution Works

What are Schemas?



How Object Name Resolution Works



Questions



Creating Data Types and Tables



Creating Data Types and Tables

- Create new data types
- Create new tables
- Create partitioned tables

Overview of Data Types

Each column, variable, expression, parameter has a data type

A data type specifies the type of data the object can hold: integers, characters, monetary data, date and time, binary

Data types are organized into the following categories:

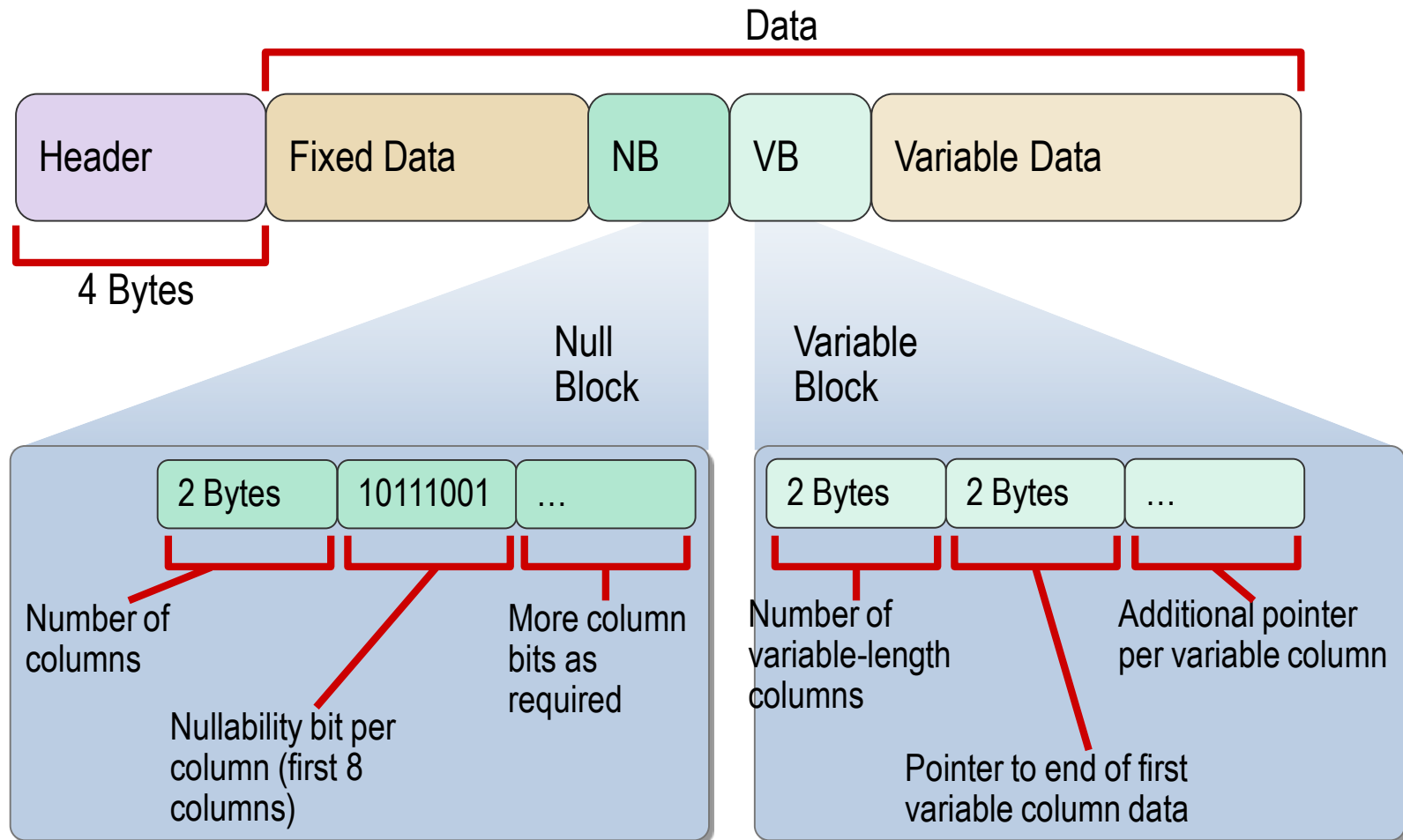
- Exact numerics
- Approximate numerics
- Date and time
- Character strings
- Unicode character strings
- Binary strings
- Other data types

New Data types in SQL2008

Creating Tables

- How SQL Server Organizes Data in Rows
- How SQL Server Organizes Large Data Values
- Types of Tables
- Considerations for Creating Tables
- Generating Transact-SQL Scripts

How SQL Server Organizes Data in Rows



Creating tables Programmatically

- ❑ Create table
- ❑ Alter table
- ❑ Drop Table

Considerations for Creating Tables

- Column names must be unique to a specific table
- You must specify a data type for each column
- You can specify in the table definition whether to allow null values in each column
- Consider the need for special columns including:
 - Computed Columns
 - Identity Property
 - timestamp Columns
 - uniqueidentifier Columns

Generating Transact-SQL Scripts

- Use Object Explorer to generate scripts for a database, or a single database object using the default options

The Generate Scripts Wizard enables you to:

- Generate scripts for multiple objects at once
- Specify various options such as permissions and collation

Sparse Columns

- Sparse columns are another addition to the SQL Server 2008 tool belt. They offer a good trade-off for many applications: taking no space if they are empty and more space if they have data. In other words they optimize storage for NULL values.
- Sparse columns are just like ordinary columns with a few limitations. They are defined with the SPARSE keyword and there is no difference on how they are used in data manipulation statements.

Self Study

- Synonyms
- DB Diagram

Questions



Implementing Data Integrity by Using Constraints and Triggers



Module Map

- **Overview of Data Integrity**
- Check Constraints
- Primary Key and Unique Constraints
- References Constraints
- Options for Enforcing Data Integrity
- Constraint Management
- **Defaults**
- Rules
- User-Defined Datatypes

Data Integrity

Data integrity helps to ensure that the data within a database is correct. This module discusses one method for enforcing the three types of data integrity

Three types of integrity help to ensure the correctness of data

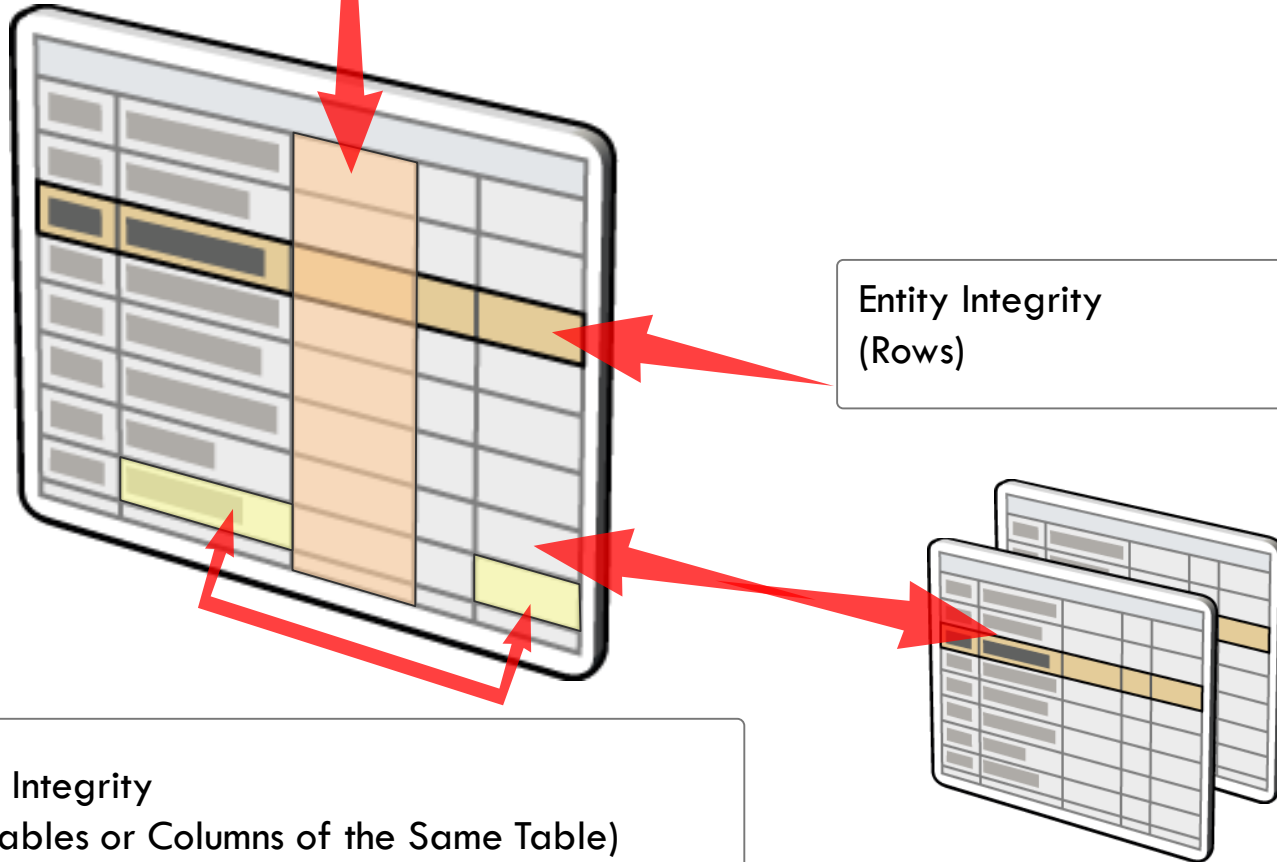
- Domain integrity
 - ▣ Ensures that any value in a column is within the acceptable set of values for that column
- Entity integrity
 - ▣ Ensures that every row of a table can be uniquely identified
- Referential integrity
 - ▣ Ensures that a given value cannot be entered in one table unless the value already exists in another table

Types of Data Integrity

Domain Integrity
(Columns)

Entity Integrity
(Rows)

Referential Integrity
(Between Tables or Columns of the Same Table)



Options for Enforcing Data Integrity

Mechanism	Description
Data types	Define the type of data that can be stored in a column
Rules	Define the acceptable values that can be inserted into a column
Defaults	Define the value of a column is a value is not specified
Constraints	Define how the Database Engine enforces data integrity
Triggers	Define code that is executed automatically when table is modified



Two Methods of Enforcement

	Domain Integrity	Entity Integrity	Referential Integrity
Constraints	Check constraints	Primary key constraints, unique constraints	References constraints
Database Objects	Rules, triggers	Indexes, triggers	Triggers

□ Focus of this module: constraints

What Are Constraints?

Integrity type	Constraint type	Description
Domain	DEFAULT	Specifies default value for column
	CHECK	Specifies allowed value for column
	FOREIGN KEY	Specifies column in which values must exist
	NULL	Specifies whether NULL is permitted
Entity	PRIMARY KEY	Identifies each row uniquely
	UNIQUE	Prevents duplication of nonprimary keys
Referential	FOREIGN KEY	Defines columns whose value must match the primary key of this table
	CHECK	Specifies the allowed value for a column based on the contents of another column

Constraint

- A constraint is an element of the **create table** statement that enforces one of the three types of data integrity
 - ▣ Declared at the column level or the table level

1)Domain Integrity

- Domain integrity ensures that any value in a column is within the acceptable set of values for that column

Check Constraint

- A check constraint enforces domain integrity during inserts and updates
 - If the statement contains a value that does not meet the check constraint, the statement fails

Column-Level Check Constraint

□ Simplified Syntax:

```
create table table_name (  
  column_name datatype [ NULL | NOT NULL | IDENTITY ]  
    [constraint constraint_name]  
    check (condition_using_column_name) ,  
  ...  
  column_name      datatype [ NULL | NOT NULL | IDENTITY ]  
)
```

□ Example:

```
create table publishers (  
  pub_id          char(4)          NOT NULL  
    constraint chk_pub_id  
    check (pub_id like "99[0-9][0-9]"),  
  pub_name        varchar(40)      NULL,  
  city            varchar(20)      NULL,  
  state           char(2)          NULL  
)
```

Column-Level Constraint Names

- If the user does not supply a name for a column-level constraint, Adaptive Server generates one in the format *table_column_systemID*

- Example:

```
create table salesdetail (  
    stor_id          char(4)          NOT NULL,  
    ord_num          varchar(20)      NOT NULL,  
    title_id         tid,  
    qty              smallint         NOT NULL  
                constraint check (qty between 1 and 3000))
```

Note: The name of the constraint is internally stored as
salesdetai_qty_768005767.

- To simplify constraint management, the user should supply a name for all constraints
 - Constraint names must be unique within a database

Table-Level Check Constraint

- Simplified syntax:

```
create table table_name (  
    column_name      datatype [ NULL | NOT NULL | IDENTITY ] ,  
    ...  
    column_name      datatype [ NULL | NOT NULL | IDENTITY ] ,  
                        [constraint constraint_name]  
                        check (condition_involving_any_column_or_columns)  
)
```

- Example:

```
create table discounts (  
    discounttype      varchar(40)                NOT NULL,  
    stor_id            char(4)                     NULL,  
    lowqty             smallint                    NULL,  
    highqty           smallint                     NULL,  
    discount           float                       NOT NULL,  
    constraint chk_low_high  
    check (lowqty <= highqty)  
)
```

Check Constraints and Defaults

- Adaptive Server allows a table to have a default and check constraint that contradict one another
- Any **insert** using the default will fail
- Example:

```
create table publishers (  
    pub_id          char(4)          NOT  
NULL,  
    pub_name        varchar(40)      NULL,  
    city            varchar(20)      default "Tulsa" NULL,  
    state           char(2)          default "OK"    NULL,  
    constraint chk_state  
    check (state in ("CA", "OR", "WA", "NV"))  
)  
  
insert into publishers (pub_id, pub_name)  
values ("9909", "Failed Imprints")
```

2) Entity Integrity

- Entity integrity ensures that every row of a table can be uniquely identified

Primary Key

- A combination of one or more columns that uniquely identifies each row in a table

Primary Key Constraint

- ❑ A primary key constraint enforces entity integrity
- ❑ It creates a unique index on the column(s) specified
- ❑ Integrity is enforced during inserts and updates
- ❑ Limit of one primary key constraint per table

Indexes Created by Primary Key Constraints

- Unique
- Clustered by default
- Do not allow NULL values in the column(s)
 - ▣ Cannot be created on nullable columns
- Cannot be dropped using the **drop index** statement

Column-Level Primary Key Constraint

□ Simplified syntax:

```
create table table_name (  
    column_name      datatype          [ NOT NULL | IDENTITY ]  
        [constraint constraint_name]  
        primary key [clustered | nonclustered],  
  
    ...  
    column_name      datatype          [ NULL | NOT NULL | IDENTITY ]  
)
```

□ Example:

```
create table publishers (  
    pub_id            char(4)           NOT NULL  
        constraint pky_c_pub_id  
        primary key,  
    pub_name          varchar(40)       NULL,  
    city              varchar(20)       NULL,  
    state             char(2) NULL  
)
```

Table-Level Primary Key Constraint

□ Simplified syntax:

```
create table table_name (  
    column_name    datatype           [ NULL | NOT NULL | IDENTITY ],  
    ...  
    column_name    datatype           [ NULL | NOT NULL | IDENTITY ],  
        [constraint constraint_name]  
        primary key [clustered | nonclustered] (column1  
[,column2...])  
)
```

□ Example:

```
create table sales (  
    stor_id        char(4)            NOT NULL,  
    ord_num        varchar(20)        NOT NULL,  
    date           datetime           NOT NULL,  
        constraint pky_c_storid_ordnum  
        primary key (stor_id, ord_num)  
)
```

Unique Constraint

- ❑ A unique constraint also enforces entity integrity
- ❑ It creates a unique index on the column(s) specified
- ❑ Integrity is enforced during inserts and updates
- ❑ Tables can have more than one unique constraint

Indexes Created by Unique Constraints

- Unique
- Nonclustered by default
- Allow for a single NULL value in the column(s)
 - ▣ Can be created on nullable columns
- Cannot be dropped using the **drop index** statement

Column-Level Unique Constraint

□ Simplified syntax:

```
create table table_name (  
    column_name      datatype      [ NULL | NOT NULL | IDENTITY ]  
        [constraint constraint_name]  
        unique [clustered | nonclustered],  
    ...  
    column_name      datatype      [ NULL | NOT NULL | IDENTITY ]  
)
```

□ Example:

```
create table publishers (  
    pub_id          char(4) NOT NULL,  
    pub_name        varchar(40) NULL  
        constraint unq_pub_name  
        unique,  
    city            varchar(20) NULL,  
    state           char(2) NULL  
)
```

Table-Level Unique Constraint

□ Simplified syntax:

```
create table table_name (  
    column_name      datatype          [ NULL | NOT NULL | IDENTITY ],  
    ...  
    column_name      datatype          [ NULL | NOT NULL | IDENTITY ],  
    [ constraint constraint_name ]  
    unique [ clustered | nonclustered ] (column1 [, column2...])  
)
```

□ Example:

```
create table sales (  
    stor_id          char(4) NOT NULL,  
    ord_num varchar(20) NOT NULL,  
    date            datetime NOT NULL,  
    constraint unq_stor_id_ord_num  
    unique (stor_id, ord_num)  
)
```

Primary Key Constraints versus Unique Constraints

	What is the default type of index?	How many NULL values does it allow?	Can you have more than one per table?
Primary Key Constraint	Clustered	None	No
Unique Constraint	Nonclustered	One	Yes

Key Constraints Summary

Primary Key Constraints:

- One PRIMARY KEY per table – based on one or more columns
- Value must be unique – null values not allowed

```
CREATE TABLE
[HumanResources].[Department] (...
CONSTRAINT
[PK_Department_DepartmentID] PRIMARY
KEY CLUSTERED ([DepartmentID] ASC)
WITH (IGNORE_DUP_KEY = OFF) ON
[PRIMARY])
```

Foreign Key Constraints:

- Referential between columns in same or different tables
- Must reference PRIMARY KEY or UNIQUE constraint

```
ALTER TABLE [Sales].[SalesOrderHeader]
WITH CHECK
ADD CONSTRAINT
[FK_SalesOrderHeader_Customer_CustomerID]
FOREIGN KEY ([CustomerID]) REFERENCES
[Sales].[Customer] ([CustomerID])
```

Summary

DEFAULT	• Defines the value of a column if a value is not specified • Only one DEFAULT per column • Some system functions allowed
CHECK	• Restricts the values that can be entered into a column • Multiple CHECK constraints allowed per column • Can reference columns in same table, no subqueries
UNIQUE	• Ensures that every value in a column is unique • Only one NULL value allowed in a unique column • Can include one or more columns

3)Referential Integrity

- Referential integrity ensures that a given value cannot be entered in one table unless it also exists in another table (or is NULL)

Foreign Key

- A foreign key is a combination of one or more columns that correspond to the primary key of another table

References Constraint

- A references constraint enforces referential integrity
- Integrity is enforced during inserts, updates, and deletes
 - ▣ If an **insert** or **update** contains foreign key values that do not exist in the primary key column(s), the statement fails
 - ▣ If an **update** or **delete** attempts to remove a primary key value that exists in a corresponding foreign key, the statement fails

Column-Level References Constraint

□ Simplified syntax:

```
create table table_name (  
    column_name      datatype          [ NULL | NOT NULL | IDENTITY ]  
        [constraint constraint_name]  
        references table_name (primary_key_column),  
  
    ...  
    column_name      datatype          [ NULL | NOT NULL | IDENTITY ]  
)
```

□ Example:

```
create table titles (  
    title_id      char(6)          NOT NULL,  
    title         varchar(80)      NULL,  
    pub_id        char(4)          NULL  
        constraint ref_pub_id  
        references publishers (pub_id),  
    notes         varchar(200)     NULL  
)
```

Table-Level References Constraint

- Simplified syntax:

```
create table table_name (  
    column_name      datatype           [ NULL | NOT NULL | IDENTITY ],  
    ...  
    column_name      datatype           [ NULL | NOT NULL | IDENTITY ],  
    [constraint constraint_name]  
    foreign key (list_of_columns_from_this_table)  
    references table_name (primary_key_column_list) )
```

- Example:

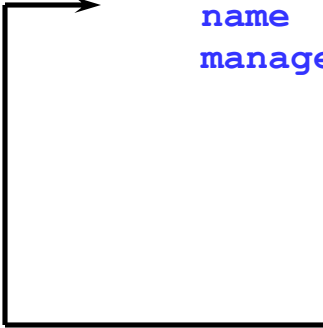
```
create table salesdetail (  
    stor_id char(4)          NOT NULL,  
    ord_num  varchar(20)     NOT NULL,  
    title_id char(6)         NOT NULL,  
    qty      smallint NOT NULL,  
    discount float          NOT NULL,  
    constraint ref_sales  
    foreign key (stor_id, ord_num)  
    references sales (stor_id, ord_num) )
```

Rules for References Constraints

- The column(s) in a parent table referred to by a references constraint in a child table must have one of the following:
 - A primary key constraint
 - A unique constraint
 - A unique index created with the **create index** command
- A references constraint can refer to columns within the same table

□ Example:

```
create table employees (  
    emp_id    char(10) NOT NULL  
             constraint pky_c_emp_id  
             primary key,  
    name      varchar(40)          NOT NULL,  
    manager   char(10) NULL  
             constraint ref_manager  
             references employees(emp_id) )
```



Restrictions on Referential Values

- Modifying primary key values:
 - ▣ Cannot update or delete if value exists in any foreign key
 - ▣ Can update or delete if value does not exist in any foreign key
- Modifying foreign key values:
 - ▣ Cannot insert or update if new value does not exist in corresponding primary key
- Dropping tables with primary keys:
 - ▣ Cannot drop table if primary key is referenced by any foreign keys

Cascading Referential Integrity



Controlled by CASCADE clause of FOREIGN KEY

Option	UPDATE Behavior	DELETE behavior
NO ACTION (default)	Return error and roll back operation	
CASCADE	Update foreign keys in referencing tables	Delete rows in referencing table
SET NULL	Set foreign keys in referencing tables to NULL	
SET DEFAULT	Set foreign keys in referencing tables to DEFAULT values	



Constraint Management

- Adding and Dropping Constraints
- Constraint Messages
- System Procedures for Constraints

Adding Constraints

□ Simplified add syntax:

```
alter table table_name
add constraint constraint_name
{ check (condition) | primary key (column_list) | unique (column_list) |
foreign key (column_list) references table (column_list) }
```

□ Examples:

```
alter table roysched
add constraint chk_hirange_lorange
check (hirange > lorange)
```

```
alter table publishers
add constraint pky_pub_id
primary key (pub_id)
```

```
alter table titles
add constraint ref_pub_id
foreign key (pub_id)
references publishers(pub_id)
```

Dropping Constraints

- Simplified drop syntax:

```
alter table table_name  
drop constraint constraint_name
```

- Examples:

```
alter table roysched  
drop constraint chk_hirange_lorange
```

```
alter table publishers  
drop constraint pky_pub_id
```

```
alter table publishers  
drop constraint unq_pub_name
```

```
alter table titles  
drop constraint ref_pub_id
```

System-Defined Constraint Messages

- System-defined constraint messages are displayed when a statement fails due to a constraint violation
- Example:

```
insert into publishers
      values ("9222", "John, Manning & Sladen",
            "Salem", "OR")
```

```
Msg  548, Level  16, State 1:
```

```
Line 1:
```

```
Check constraint violation occurred, dbname =
      'pubs2', table name = 'publishers',
      constraint name = 'chk_pub_id'.
```

User-Defined Constraint Messages

- A user-defined constraint message is displayed when a statement fails due to a check or references constraint violation
 - ▣ The system-defined constraint message is not displayed

- Example:

```
insert into publishers
  values ("9222", "John, Manning & Sladen",
         "Salem", "OR")
```

```
Msg 30001, Level 16, State 1:
```

```
Line 1:
```

```
The publisher ID must be in the format 99xx.
Each x must be a digit 0 through 9.
```

Creating and Binding Messages

- Constraint messages can be created and bound to the relevant constraint(s)
 - ▣ Messages can be bound to multiple constraints
- Simplified create syntax:
`sp_addmessage message_number, "message_text"`
- Example:
`sp_addmessage 30001, "The publisher ID must be in
the format 99xx. Each x must be a digit 0
through 9."`
- Simplified bind syntax:
`sp_bindmsg constraint_name, message_number`
- Example:
`sp_bindmsg chk_pub_id, 30001`

Unbinding and Dropping Messages

- Simplified unbind syntax:

`sp_unbindmsg constraint_name`

- Example:

`sp_unbindmsg chk_pub_id`

- Simplified drop syntax:

`sp_dropmessage message_number`

- Example:

`sp_dropmessage 30001`

System Procedures for Constraints

- **sp_helpconstraint** *table_name*
 - ▣ Displays information about the constraints on the specified table
- **sp_rename** *old_constraint_name*,
new_constraint_name
 - ▣ Changes the name of a check or references constraint

Default

- A default is a database object that supplies a value to a column during an **insert** statement if no value was specified for that column
- It can be bound to one or more columns

Creating and Binding Defaults

- Simplified Create Syntax:

`create default default_name as
constant_expression`

- Example:

`create default def_state as
"CA"`

- Simplified Bind Syntax:

`sp_bindefault default_name, object_name`

- Example:

`sp_bindefault def_state, "publishers.state"`

Rules for Binding Defaults

- A column can have only one default bound to it
 - ▣ If you attempt to bind a default to a column that already has a default bound to it, the attempted binding fails
- A column with a **default** clause cannot have a default bound to it
- When a default is bound, existing data in the table is not affected
 - ▣ It applies only to data inserted after the default was bound

Unbinding and Dropping Defaults

- Simplified unbind syntax:

`sp_unbinddefault object_name`

- Example:

`sp_unbinddefault "publishers.state"`

- Simplified drop syntax:

`drop default default_name`

- Example:

`drop default def_state`

Rule

- A rule is a database object that prevents inserts and updates if the rule's condition is not met
 - ▣ Like a check constraint, it can be used to enforce domain integrity
- A rule can be bound to one or more columns

Creating Rules

□ Simplified Syntax:

```
create rule rule_name as  
    condition_expression
```

□ Examples:

```
create rule rul_state as  
    @state in ("CA", "CO", "WA")
```

```
create rule rul_discount_range as  
    @discount between 0 and 20
```

```
create rule rul_file_date as  
    @date < 12 or @date > 22
```

```
create rule rul_pub_id as  
    @pub_id in ("1389", "0736", "0877") or  
    @pub_id like "99[0-9][0-9]"
```

Binding Rules

- Simplified syntax:

`sp_bindrule rule_name, object_name`

- Example:

`sp_bindrule rul_state, "publishers.state"`

- A column can have only one rule bound to it

- ▣ If you bind a rule to a column, any previously bound rule is unbound
- ▣ This is the opposite of what happens when you try to bind a default to a column that already has a default

- A column can have both a constraint and a rule bound to it

- ▣ Values must match both the constraint and the rule to be inserted or updated

- When a rule is bound, existing data in the table is not affected

- ▣ It applies only to data inserted after the rule was bound

Unbinding and Dropping Rules

- Simplified unbind syntax:

`sp_unbindrule object_name`

- Example:

`sp_unbindrule "publishers.state"`

- Simplified drop syntax:

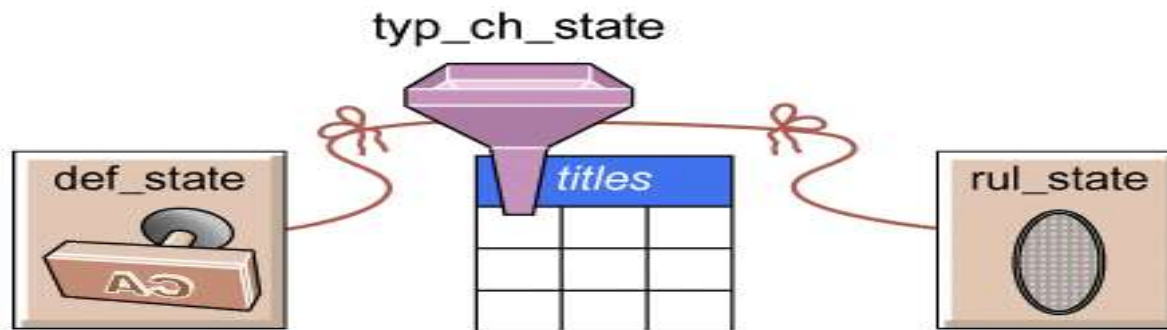
`drop rule rule_name`

- Example:

`drop rule rul_state`

User-Defined Datatypes

- Defaults and rules can be bound to user-defined datatypes



- Columns using the datatype use the default or rule
- If the default or rule changes, the change is reflected in all instances of any user-defined datatype bound to that default or rule
 - ▣ Existing data is not affected by the change

Binding and Unbinding to User-Defined Datatypes

- Binding and unbinding objects to user-defined datatypes uses the same syntax as binding and unbinding objects to table columns

- Default example:

```
sp_bindefault def_state, typ_ch_state
```

```
sp_unbindefault typ_ch_state
```

- Rule example:

```
sp_bindrule rul_state, typ_ch_state
```

```
sp_unbindrule typ_ch_state
```

Using Bound User-Defined Datatypes

- Once a user-defined datatype has a default and/or rule bound to it, any column using that user-defined datatype automatically inherits the default and/or rule
- Example:

```
sp_bindefault def_state, typ_ch_state
```

```
create table publishers (  
...  
state                typ_ch_state  
...)
```

```
create table authors (  
...  
state                typ_ch_state  
...)
```

Column and Datatype Precedence

- If a column:
 - ▣ Has a rule bound to it, *and*
 - ▣ Uses a user-defined datatype with a different rule bound to it, *then*
 - ▣ The rule bound to the column takes precedence

System Procedures for Defaults and Rules

- **sp_helptext** { *default_name* | *rule_name* }
 - ▣ Displays the text used to create the specified default or rule
- **sp_rename** *old_object_name*, *new_object_name*
 - ▣ Changes the name of a default or rule

Summary of Database Objects

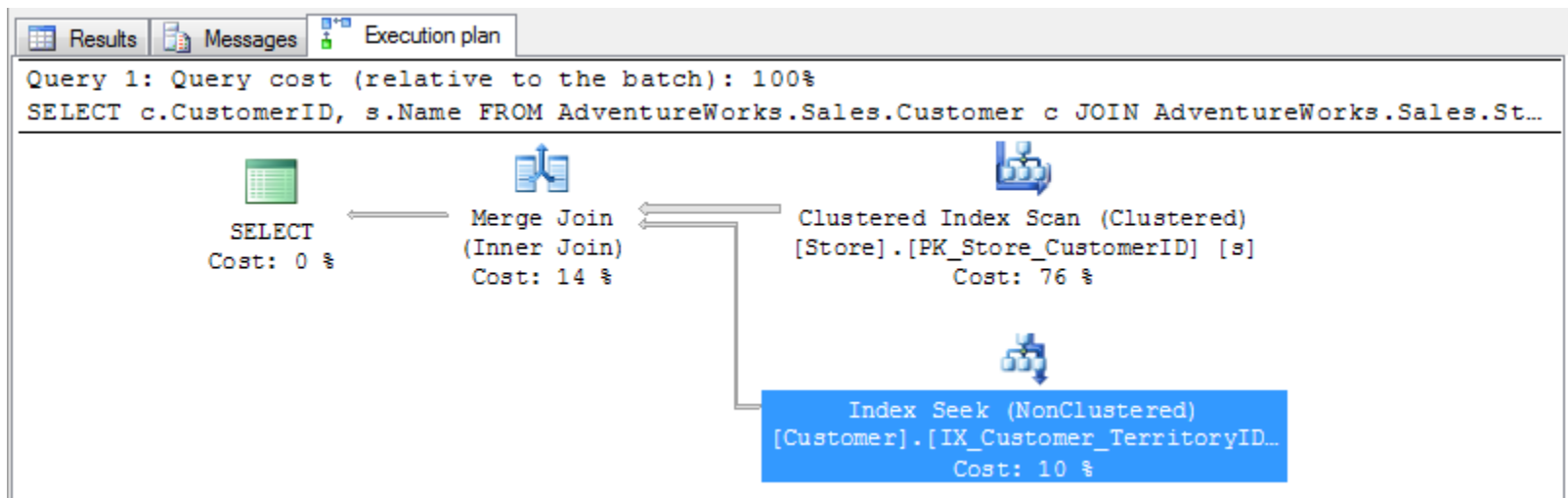
Object Type	Create	Drop	Examine	Bind/Unbind
User-Defined Datatype	sp_addtype	sp_droptype	sp_help	not applicable
Table	create table select into	drop table	sp_help	not applicable
View	create view	drop view	sp_help sp_helptext	not applicable
Index	create index	drop index (drop table)	sp_help sp_helpindex	not applicable
Default	create default	drop default	sp_helptext	sp_bindefault sp_unbindefault
Rule	create rule	drop rule	sp_helptext	sp_bindrule sp_unbindrule

Execution Plans

- Shows how the Database Engine navigates tables and indexes
- View Estimated Execution Plan before query execution
- View Actual Execution Plan after execution

Generate Execution Plans with:

- SQL Server Management Studio
- T-SQL SET options
- SQL Server Profiler



What Are Correlated Subqueries?

1

Outer query passes column values to the inner query

2

Inner query uses that value to satisfy the inner query

```
USE northwind
SELECT orderid, customerid
FROM orders AS or1
WHERE 20 < (SELECT quantity
            FROM [order details] AS od
            WHERE or1.orderid = od.orderid
            AND   od.productid = 23)

GO
```

3

Inner query returns a value back to the outer query

4

The process is repeated for the next row of the outer query



Back to Step 1

Building a Correlated Subquery

Outer Query

```
SELECT c.LastName, c.FirstName  
FROM Person.Person c  
JOIN HumanResources.Employee e  
ON e.BusinessEntityID =  
   c.BusinessEntityID
```



Inner Query

```
SELECT Bonus  
FROM Sales.SalesPerson
```

Correlated Subquery

```
SELECT c.LastName, c.FirstName  
FROM Person.Person c JOIN HumanResources.Employee e  
ON e.BusinessEntityID = c.BusinessEntityID  
WHERE 5000.00 IN  
   (SELECT Bonus  
    FROM Sales.SalesPerson sp  
    WHERE e.BusinessEntityID = sp.BusinessEntityID) ;
```



Using Correlated Subqueries

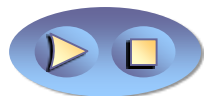
- Correlated subqueries are executed repeatedly, once for each row that may be selected by the outer query

```
SELECT DISTINCT c.LastName, c.FirstName
FROM Person.Person c JOIN HumanResources.Employee e
ON e.BusinessEntityID = c.BusinessEntityID
WHERE 5000.00 IN
(SELECT Bonus
FROM Sales.SalesPerson sp
WHERE e.BusinessEntityID = sp.BusinessEntityID);
```

Result Set:

LastName	FirstName
Ansman-Wolfe	Pamela
Saraive	José

(2 row(s) affected)



Using the EXISTS Clause with Correlated Subqueries

- When a subquery is introduced with the keyword EXISTS, the subquery functions as an existence test

```
SELECT Name
FROM Production.Product
WHERE EXISTS
    (SELECT * FROM Production.ProductSubcategory
     WHERE ProductSubcategoryID =
        Production.Product.ProductSubcategoryID
     AND Name = 'Wheels')
```

Result Set:

Name
LL Mountain Front Wheel
ML Mountain Front Wheel

...
(14 row(s) affected)

